ZEGHOUDI Soleyman

05/01/2026-06/02/2026

43 bis avenue du Maréchal Foch

Rapport de stage 2025-2026

Introduction:

During my internship at PinkTech, I was responsible for the digital transformation of "Maison Lutèce." The main objective was to create a web application that would enable the company to manage its catalog of high-end services, its members, and its reservations.

For this project, I used the symfony 7 framework, as this is the one used in the company. In order to quickly provide an administration panel, I integrated the easyadmin bundle.

This report focuses on backend development. I will explain how I designed the database, created entities using the command line interface, and configured the administration dashboard. I will also detail the technical issues I encountered, particularly with regard to routing and controllers, and how I resolved them.

L’entreprise PinkTech est une micro-entreprise spécialisé dans le développement de sites et d'applications web sur-mesure, elle t'accompagne de l'idée jusqu'à la réalisation de ton projet en utilisant Symfony/Twig/CSS et JS.J’ai pour seul outil un pc qui m’a été fournis par l’entreprise .

Au début de mon stage, j'ai disposé d'une période d'autonomie j’ai choisi de mettre ce temps à profit pour avancer significativement sur la conception et le développement de mon portfolio professionnel, support indispensable pour les épreuves du BTS SIO.

Cette démarche m'a permis de rester productif et d'organiser mon environnement de travail efficacement avant de me consacrer pleinement au projet de la "Maison Lutèce".

Avant de lancer le développement sous symfony, une phase d'analyse a été nécessaire pour structurer les informations de la conciergerie. L'enjeu était de traduire des besoins métier complexes (gestion de services variés, tarification, rôles utilisateurs) en un schéma de base de données relationnel cohérent.

J'ai utilisé l'outil notion pour faire un brouillon des tables. J'ai commencé par définir la table Utilisateur, qui est centrale au système. Elle doit permettre de gérer à la fois les clients, les administrateurs et les prestataires via un système de rôles .

Utilisateur	Formats
id	int
nom	Varchar
prenom	Varchar
email	Varchar
mdp	Text
role	varchar

Ensuite, j'ai structuré le catalogue de services. Pour permettre une évolution future, j'ai séparé les services de leurs catégories. Ainsi, la table services contient les détails de l'offre tandis que categories services permet de les classer.

Categorie_services	formats
id	int
nom	Varchar

Services	Formats
id	int
utilisateurs_id	int
categorie_id	int
nom	Varchar
prix	float
description	text

Pour répondre aux exigences d'une conciergerie de luxe, le modèle de données devait aller plus loin qu'un simple catalogue. J'ai dû intégrer la gestion de la relation client et la facturation.

J'ai conçu les tables messages et paiements .

messages	formats
id	int
expediteur_id	int
contenue	text
date	timestamp

paiements	format
id	int
reservation_id	int
commission	decimal
net_partenaire	int

Enfin, la table la plus critique du système est la reservation. C'est une table pivot qui relie un client à un service, tout en stockant le statut de la commande et la date.

Reservation	format
id	int
client_id	int
service_id	int
statut	Varchar
date	Timestamp
prix	decimal

Une fois la conception validée, je suis passé à la phase de réalisation technique. J'ai utilisé le maker bundle de Symfony via le terminal. Cette approche permet de générer des classes PHP en respectant les standards PSR.

Pour chaque table définie lors de la conception j'ai utilisé la commande php bin/console make:entity. L'étape la plus délicate a été la configuration des relations entre les entités.

Par exemple pour la table reservation j'ai dû définir une relation ManyToOne avec services .

```
 What class should this entity be related to?:
 > Services
Services

What type of relationship is this?
 ------------ ---------------------------------------------------------------
------------
  Type         Description

 ------------ ---------------------------------------------------------------
------------
  ManyToOne    Each Reservation relates to (has) one Services.

               Each Services can relate to (can have) many Reservation objects
.


  OneToMany    Each Reservation can relate to (can have) many Services objects
.
               Each Services relates to (has) one Reservation.


  ManyToMany   Each Reservation can relate to (can have) many Services objects
.
               Each Services can also relate to (can also have) many Reservati
on objects.


  OneToOne     Each Reservation relates to (has) exactly one Services.

               Each Services also relates to (has) exactly one Reservation.

 ------------ ---------------------------------------------------------------
------------

 Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]:
 > ManytoOne
```

Lors de cette phase, j'ai été confronté à des erreurs de syntaxe, notamment en tentant de créer l'entité Photo. J'avais entré une commande alors que le terminal attendait un nom de propriété.

```
 [ERROR] "php bin/console make:entity Photo" is not a valid PHP property name.
ted, please modify it to your needs

r_messages');
 Add another property? Enter the property name (or press <return> to stop addi
ng fields):
 >
```

Cette erreur m'a permis de mieux comprendre le fonctionnement interactif du Maker Bundle et l'importance de la rigueur dans la saisie des commandes et de ne pas ce precipiter comme je le faisais.

Les fichiers PHP générés dans src/Entity ne sont qu'une représentation objet des données. Pour créer physiquement les tables, j'ai utilisé l'ORM Doctrine et son système de migrations.

J'ai exécuté la commande php bin/console doctrine:migrations:migrate.

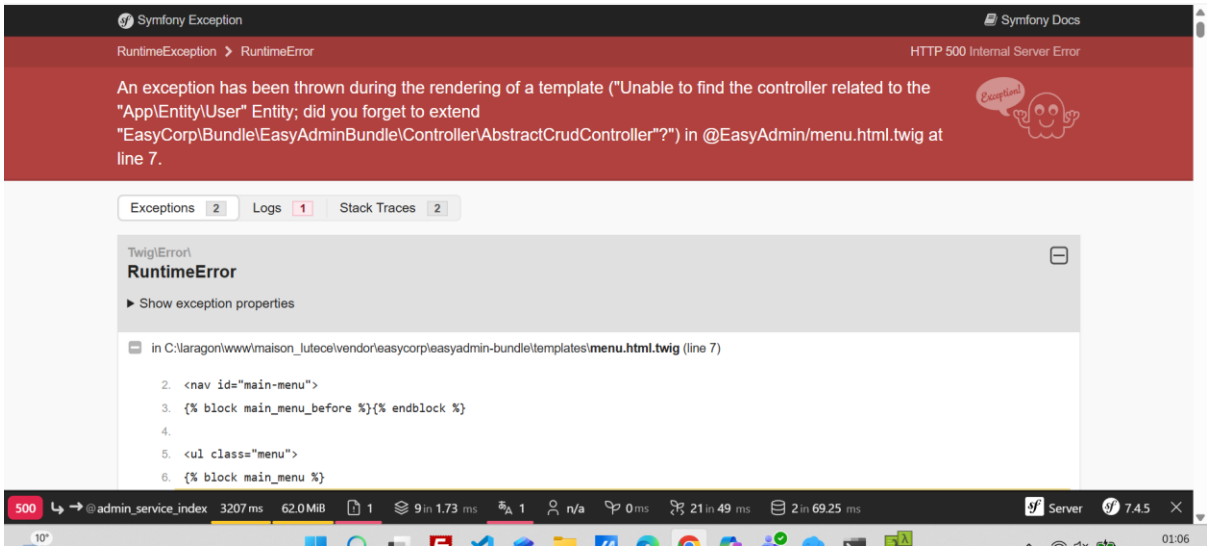
```
C:\laragon\www\maison_lutece(master)
λ php bin/console doctrine:migrations:migrate

 WARNING! You are about to execute a migration in database "maison_lutece" tha
t could result in schema changes and data loss. Are you sure you wish to conti
nue? (yes/no) [yes]:
 > yes
de la page HTML sous forme d'objets. JavaScript peut :
[notice] Migrating up to DoctrineMigrations\Version20260212080056
[notice] finished in 2823.2ms, used 24M memory, 1 migrations executed, 7 sql q
ueries

 [OK] Successfully migrated to version: DoctrineMigrations\Version202602120800
56


C:\laragon\www\maison_lutece(master)
λ
```

J'ai choisi easyadmin pour générer automatiquement l'interface CRUD

J'ai configuré le contrôleur principal DashboardController.php pour définir le titre du site et structurer le menu latéral.

```php
class DashboardController extends AbstractDashboardController
    6 references | 0 overrides
    public function index(): Response
    {
        return parent::index();
    }

    0 references | 0 overrides
    public function configureDashboard(): Dashboard
    {
        return Dashboard::new()
            ->setTitle(title: '<b>Maison Lutèce</b> - Administration')
            ->setFaviconPath(path: 'favicon.ico');
    }

    0 references | 0 overrides
    public function configureMenuItems(): iterable
    {
        yield MenuItem::linkToDashboard(label: 'Tableau de bord', icon: 'fa fa-home');

        yield MenuItem::section(label: 'Catalogue');
        yield MenuItem::linkToCrud(label: 'Nos Services', icon: 'fas fa-concierge-bell', entityFqcn: S

        yield MenuItem::section(label: 'Utilisateurs');
        yield MenuItem::linkToCrud(label: 'Membres', icon: 'fas fa-users', entityFqcn: User::class);
    }
}
```

Lors de la première tentative d'accès au menu utilisateurs j'ai rencontré une erreur HTTP 500.



L'analyse de l'erreur m'a permis de comprendre que j'avais créé un lien vers l'entité utilisateur dans le menu, mais que je n'avais pas généré le CRUD Controller correspondant. Easyadmin ne savait donc pas comment afficher cette entité.

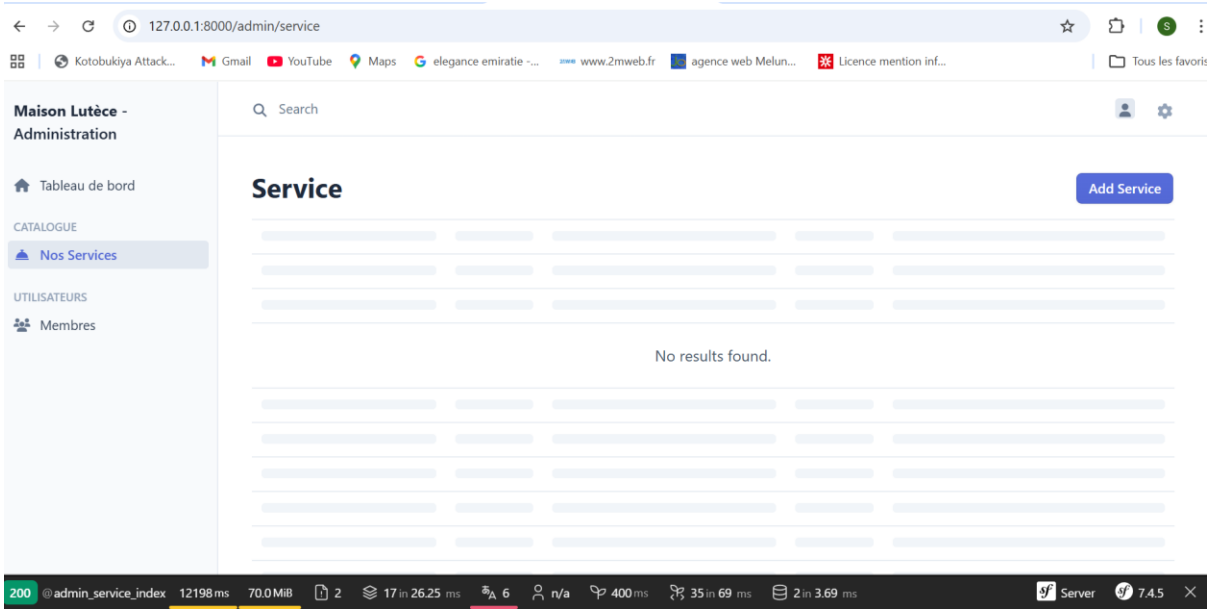
J'ai corrigé le problème en générant le contrôleur manquant via la commande php bin/console make:admin:crud.

```
C:\laragon\www\maison_lutece(master)
λ php bin/console make:admin:crud

 Which Doctrine entity are you going to manage with this CRUD controller?:
  [0] App\Entity\Service
  [1] App\Entity\User
 > User
User

 [ERROR] Value "User" is invalid

 Which Doctrine entity are you going to manage with this CRUD controller?:
  [0] App\Entity\Service
  [1] App\Entity\User
 > 1
1

 Which directory do you want to generate the CRUD controller in? [src/Controller/Admin/]:
 >

 Namespace of the generated CRUD controller [App\Controller\Admin]:
 >

 [OK] Your CRUD controller class has been successfully generated.

 Next steps:
 * Configure your controller at "src/Controller/Admin/UserCrudController.php"
 * Read EasyAdmin docs: https://symfony.com/doc/master/bundles/EasyAdminBundle/index.html

C:\laragon\www\maison_lutece(master)
λ
```

Une fois tous les contrôleurs générés j'ai pu finaliser l'organisation du menu dans le code, en regroupant les liens par sections logiques pour améliorer l'expérience utilisateur .

```php
class DashboardController extends AbstractDashboardController
    public function index(): Response
    {
        return parent::index();
    }

    0 references | 0 overrides
    public function configureDashboard(): Dashboard
    {
        return Dashboard::new()
            ->setTitle(title: '<b>Maison Lutèce</b> - Administration')
            ->setFaviconPath(path: 'favicon.ico');
    }

    0 references | 0 overrides
    public function configureMenuItems(): iterable
    {
        yield MenuItem::linkToDashboard(label: 'Tableau de bord', icon: 'fa fa-home');

        yield MenuItem::section(label: 'Catalogue');
        yield MenuItem::linkToCrud(label: 'Nos Services', icon: 'fas fa-concierge-bell', entityFqcn: Service::class);

        yield MenuItem::section(label: 'Utilisateurs');
        yield MenuItem::linkToCrud(label: 'Membres', icon: 'fas fa-users', entityFqcn: User::class);
    }
}
```

Le résultat est une interface d'administration complète et fonctionnelle :



Cette étape du projet m'a permis de consolider mes compétences sur le framework Symfony et l'architecture MVC dont j’avais été former lors de mon précèdent stage . J'ai réussi à passer d'une modélisation théorique à une application fonctionnelle, en surmontant les difficultés techniques liées aux migrations et au routage des contrôleurs.

J’ai donc réalisé pour l’entreprise l'architecture Backend et la base de données du projet "Maison Lutèce". Ce stage m'a permis de savoir comment mieux s’organiser dans son travail et se répartir les tâches et utiliser son autonomie à bon escient, notamment grâce à l'utilisation d'outils comme Notion. J'ai surtout compris la méthodologie de création d'un projet : comment partir d’une simple idée pour en déduire les fonctionnalités techniques qu’on va vouloir instaurer, et définir précisément de quelles tables nous aurons besoin dans la base de données.

Ce stage m'a également permis de pratiquer concrètement le framework Symfony, dont on avait commencer a utiliser en cours